Itara: Paying for a Team's Context Once

A Self-Hosted Open-Weight Gateway with Shared Memory, Permission-Scoped KV Caching, and Confidence-Gated Escalation to a Closed Model

Prince Mutegetsi
pmuteget@caltech.edu

November 2026

Abstract

Teams that use a closed large language model pay for the same context again and again: every engineer sends the same codebase, documents and decisions with every question. We present **Itara**, a self-hosted gateway that a whole team shares. An open-weight model (Llama 3.1 8B) answers the questions it is confident about. A team memory, compressed once and isolated per team by row-level security, stands in for raw context. A KV cache, keyed by permission, lets teammates reuse each other's prefill while preventing cross-team timing leaks. A cascade gate built on the open model's own confidence decides when to escalate to the closed model (Muse Spark). We evaluate it on the real models with 300 SWE-bench Lite localisation tasks and 300 LoCoMo questions. On code, the closed model given the team's memory summaries reads 8.9% of the context and matches its full-context accuracy (86.3% vs. 86.0%) at 23.0% of the tokens, about a quarter; what remains of the bill is the closed model's own reasoning. The open model's confidence separates its right answers from wrong ones (AUC 0.68 on code, 0.80 on conversations), and the gate beats random escalation by 5 points at a 10% budget on code. Shared memory cuts ingest work per engineer by 35% at 16 engineers, against 13% without sharing, and the permission-scoped cache lets teammates reuse prefill while other teams reuse none. On conversations, retrieval rather than routing limits accuracy. Before any experiment we also set an ambitious goal: match the closed model within 3 points while sending it at most 10% of its tokens. It is not reached yet (at 10%, 74.0% on code against 86.0%), but an oracle router passes on code (90.3% at 8.8%), and at minimal reasoning effort, a setting chosen after the run, the gate comes within about 2 points. Runs added after the results with other open models drafting raise the open model's own accuracy on code from 55.7% to 67.0% (Llama 3.3 70B) and 76.0% (Muse Glimmer 30B, Meta's open reasoning model), and the cascade at 10% to 77.7% and 80.3%; a perfect judge of either model's drafts would pass the goal, and Glimmer's reasoning makes its drafts several times slower. A better draft judge, more training data and better retrieval are the next steps. Post-hoc analyses are reported separately and with intervals.

1 Introduction

A software team that adopts a frontier model through an API pays per token, and most of those tokens are context: the files around an issue, the design notes, the decisions made last month. The same context is sent by every engineer, for every question, and paid for each time. Two developments make a different arrangement possible. Open-weight models are now good enough to answer many routine questions themselves [18], and serving systems can reuse the attention state of text that has already been processed [5, 14, 17, 29, 31]. A team could therefore process its shared knowledge once, answer what it can locally, and send the closed model only what it must, in compact form.

Each of these ideas has been studied alone: routers and cascades between cheap and expensive models [1, 3, 7, 22], retrieval and memory for language models [15, 23], and KV-cache reuse in serving [14, 29]. They have not, to our knowledge, been combined and measured as one shared service for a team, where the access-control questions that a shared cache raises [6, 24] must be answered as part of the design.

The goal, stated in advance. Before any experiment we wrote down a pass rule: *for a team sharing one codebase, the gateway matches the accuracy of sending context straight to the closed model, within 3 points, while sending it at most 10% of the closed model's tokens.* We report what the system achieves and then the verdict on that rule, and keep every choice made after seeing results (larger budgets, a lower reasoning effort) separate and labelled.

Contributions.

- **A system.** Itara is an OpenAI-compatible gateway (Section 3) combining a per-team compressed memory under Postgres row-level security, a KV cache whose keys include the reader's permission scope, a confidence-gated cascade to a closed model, and exact token accounting with a hard spending cap.
- **A pre-registered evaluation** on the real models: all 300 SWE-bench Lite issues [10] as a file-localisation task and 300 LoCoMo questions [19], against six baselines and an oracle router, with out-of-fold gates and paired bootstrap

intervals (Section 4).

- **What works.** Compressed team memory lets the closed model match its full-context accuracy on code with under a tenth of the context. The open model's confidence is a usable escalation signal. Sharing memory across a team reduces per-engineer work as the team grows, and scoping the cache by permission removes the cross-team timing channel by construction.
- **An honest test against an ambitious goal.** The 10% goal is not reached yet on either benchmark. We show why, and what would close the gap: with a reasoning model, the closed model's own output, not the context, sets the cost of an escalation; the 8B open model's accuracy limits any draft-based gate; and on conversations, retrieval caps every routing policy (Sections 5 and 6).

2 Related work

Routing and cascades. FrugalGPT [1] chains models of increasing cost and stops when a scorer accepts an answer. Hybrid LLM [3] and RouteLLM [22] learn a router that predicts, before generation, whether the small model suffices. Gupta et al. [7] show that sequence-level confidence is length-biased and that token-level quantiles defer better. Itara is a cascade in this sense, but its unit of saving is different: escalations carry compact team memory instead of raw context, and the gate is scored against token budgets measured on the closed model's actual usage, including its reasoning tokens.

Knowing when a model is wrong. Models' own probabilities carry information about correctness [11]. Agreement between sampled answers is a strong black-box signal [13, 20, 26]. Our cascade gate combines token log-probabilities, agreement among four resamples and a yes/no self-check, all computed on the open model's cached prefix.

Memory and context. Retrieval-augmented generation [15] and agent memory systems such as MemGPT [23] keep knowledge outside the prompt. Long prompts are not only expensive but can hurt accuracy [8, 16], and prompt compression reduces tokens directly [9]. Itara's memory is per team rather than per user, is compressed once by the open model, and is retrieved by hybrid vector and keyword search fused by reciprocal rank [2].

KV-cache reuse and its risks. vLLM's paged attention and prefix caching [14], SGLang's radix tree [31], Prompt Cache [5], CacheBlend [29] and LMCache [17] reuse attention state across requests. Shared caches leak: response timing reveals whether another user's prompt was cached [24], and audits found cross-user cache sharing at several commercial APIs [6]. Itara keys the cache by permission scope so that a hit is only possible between readers with the same access.

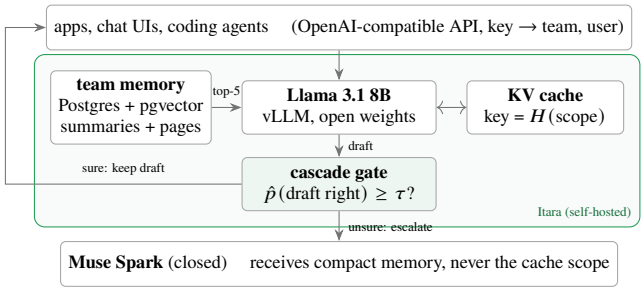


Figure 1: Itara. The open model drafts every answer with the team's memory, reusing the team's cached prefill. The gate keeps confident drafts and escalates the rest to the closed model with compact memory. Row-level security and scope-keyed caching isolate teams.

3 System

Itara is a single server (Figure 1) that speaks the OpenAI chat-completions API, so any client adopts it by changing a base URL. It runs the open model on vLLM [14], stores memory in Postgres with pgvector [12], and reaches the closed model through the Meta Model API [21].

Identity and isolation. Each API key maps to one team and one user, and only its hash is stored; identity never comes from client headers. Every memory row carries a team and an optional owner. Postgres row-level security [25], forced even for the table owner, restricts every query to rows of the caller's team that are shared or owned by the caller. Personal memory is private by default; code, documents and decisions are shared with the team.

Shared memory. A document is split on blank lines into pages of at most 2,000 characters. The open model compresses each page into a JSON summary written as one fact per sentence (who did what, when) plus keywords; if it fails twice, the page's opening text is stored instead. Summaries are embedded with BGE-small [28]. Raw pages are kept behind their summaries. A question retrieves its top-k=5 summaries by reciprocal rank fusion [2] of a cosine-similarity ranking and a full-text ranking:

$$\text{score}(m) = \sum_{r \in \{\text{vec},\text{text}\}} \frac{1}{60 + \text{rank}_r(m)}, \qquad (1)$$

with ties broken by id so retrieval is deterministic. Code sources are labelled with their commit; conversation sources with their session date, which the compression prompt is told to use when resolving relative times.

Permission-scoped KV cache. A request to the open model carries a cache salt $s = H(\text{team})$ if all retrieved memory is shared, or $H(\text{team} \,\|\, \text{user})$ if any is private. vLLM mixes the salt into the hash of the first KV block, so prefix hits occur only between requests with the same scope.

A teammate asking about the same code reuses the prefill; a member of another team, even with an identical prompt, never hits the cache and cannot learn from its timing [24]. The closed model receives memory but never the salt.

Routes. Each question takes one route: *local_memory* (the open model answers with the top-5 summaries), *escalate_compact* (the closed model answers with the same summaries), or *escalate_raw* (the closed model answers with the raw pages behind them). Requests with tool calls pass to the closed model unchanged.

Gates. A *rule gate* escalates questions longer than 200 tokens or containing a fenced code block. A *per-route gate* fits, for each route, a logistic model of $P(\text{right} \mid x)$ on 18 features of the question, the retrieved memory and the open model's draft, and picks

$$\hat{r}(x) = \arg\max_r \ \hat{P}_r(\text{right} \mid x) - \lambda \widehat{\text{cost}}_r(x), \qquad (2)$$

where the cost is estimated in dollars before the call; sweeping λ traces a cost–accuracy curve. Training labels come from escalation regret: each route's actual correctness on the item. The *cascade gate*, which Itara ships, is simpler. The open model always drafts first. A single logistic score $\hat{p}$ of "the draft is right" is computed from six confidence features: the mean and minimum token log-probability of the draft, the share of tokens with probability below one half, agreement of four resamples at temperature 1 with the draft (cosine similarity and exact first-line match), and the probability of "yes" when the model is asked whether its answer is correct and supported by memory. The resamples and the self-check reuse the draft's cached prefix, so they cost little prefill. If $\hat{p} < \tau$ the question escalates: with summaries when most retrieved memory is code, with raw pages otherwise.

Accounting and the cap. Every model call is recorded with the token counts the model reports, tagged with the request it belongs to. Before a paid call, its worst-case cost is reserved under a lock against a hard cap, so concurrent requests cannot exceed it. An admin view shows, per person and team, the routes taken, cached tokens reused and spend (a live topology dashboard and a command-line tool are part of the release).

4 Experimental setup

Models and infrastructure. The open model is Llama 3.1 8B Instruct [18] on vLLM with prefix caching on one H100 (Modal). The closed model is Muse Spark 1.2 through the Meta Model API [21], a reasoning model whose output tokens include its reasoning, at its default effort unless stated. Answers are generated at temperature 0.

Benchmarks. *Code*: all 300 SWE-bench Lite issues [10], posed as file localisation, a step used by recent agent pipelines [27]: name at most five files to change. An item counts as right only if every file in the gold patch is named; no judge is needed. Each repository is a team; its memory is the files of the BM25-retrieved context distributed with the benchmark (about 13K tokens per issue). *Conversations*: 300 questions from LoCoMo [19] (categories 1–4, sampled with a fixed seed), each conversation a team and each session a memory source. Answers are judged by the closed model at minimal effort [30]; a hand check of 50 randomly sampled verdicts across conditions agreed with the judge on 48.

Conditions. *Muse, full context* sends the closed model the whole retrieved code (on average 11,253 prompt tokens) or the whole conversation (18,328); it is the accuracy reference and the token denominator. *Always escalate* (summaries or raw pages), *rule gate*, *Llama + memory* (never escalate), and Itara's *cascade gate* complete the set. The *oracle router* knows every route's correctness and picks the cheapest right route; no gate choosing among these routes can beat it.

Metrics and protocol. Accuracy is the share of items right. Cost is the closed model's tokens (prompt plus completion) as a share of the reference's on the same items; we also report the prompt-only share. Gates are trained and scored out of fold: five folds grouped by team, so a repository or conversation is never in both training and test. Intervals on the accuracy gap are paired bootstrap 95% intervals over items (4,000 resamples) [4]. The pass rule is applied to point estimates at each benchmark's 10% operating point.

Team replay. To measure sharing, SWE-bench tasks on one repository are replayed by $n \in \{1, 2, 4, 8, 16\}$ simulated engineers with two tasks each, either as one team (shared memory and cache) or each alone. Tasks are ordered either by overlap with files the team already touched (*related*) or at random. Pages ingested per engineer is the timing-independent measure of the gateway's work.

5 Results

5.1 On code, compact memory matches the closed model

Table 1 and Figure 2 summarise every condition. Sending every question to the closed model with the top-5 memory summaries scores 86.3% against 86.0% (gap +0.3, interval −2.3 to +3.0): no measurable loss. The closed model then reads 8.9% of the prompt tokens it would otherwise receive (1,004 instead of 11,253 per question). Raw pages behind the summaries reach 87.0% at 26.0% of the prompt. Yet the total-token share of the summary route is 23.0%, not 9%: Muse writes about 2,822 reasoning and answer tokens per escalation however short its prompt. With a reasoning model,

Table 1: Main results. Gap: accuracy minus Muse with the full context, in points, with a paired bootstrap 95% interval. Token share: Muse prompt + completion tokens relative to the reference on the same items; prompt share counts prompt tokens only. Kept: questions answered by the open model. On LoCoMo the rule gate never escalates (the questions are short and contain no code blocks), so it matches Llama + memory.

Condition	Accuracy (%)	Gap (pts) [95% CI]	Token share (%)	Prompt share (%)	Kept (%)
SWE-bench Lite (code), 300 items					
Muse, full context	86.0	–	100.0	100.0	0
Itara, cascade gate at 10%	74.0	−12.0 [−17.0, −7.3]	10.0	3.6	61
Muse + raw pages (always)	87.0	+1.0 [−1.3, +3.3]	34.1	26.0	0
Muse + summaries (always)	86.3	+0.3 [−2.3, +3.0]	23.0	8.9	0
Rule gate	82.7	−3.3 [−6.7, +0.0]	21.0	8.0	17
Llama + memory (never escalate)	55.7	−30.3 [−36.3, −24.7]	0.0	0.0	100
Oracle router (upper bound)	90.3	+4.3	8.8	3.6	65
LoCoMo (conversations), 300 items					
Muse, full context	87.0	–	100.0	100.0	0
Itara, cascade gate at 10%	54.7	−32.3 [−38.0, −26.3]	9.4	6.6	41
Muse + raw pages (always)	62.7	−24.3 [−29.7, −19.0]	15.7	11.3	0
Muse + summaries (always)	46.0	−41.0 [−47.0, −35.0]	7.7	2.6	0
Rule gate	38.3	−48.7 [−54.7, −42.7]	0.0	0.0	100
Llama + memory (never escalate)	38.3	−48.7 [−54.7, −43.0]	0.0	0.0	100
Oracle router (upper bound)	66.7	−20.3	3.6	2.3	72

Table 2: How well each signal separates the open model's right drafts from wrong ones (AUC, out of fold; 0.5 is chance).

Signal	Code	Conv.
4 resamples give the draft's answer	0.68	0.71
Similarity of 4 resamples to the draft	0.60	0.70
Mean token log-probability	0.60	0.74
Minimum token log-probability	0.58	0.70
Self-check: P(yes) that the draft is right	0.61	0.66
Best question–memory similarity	0.63	0.55
Cascade score (6 confidence features)	0.65	0.74
Per-route gate, P(draft right) (18 features)	0.68	0.80

Table 3: Gates at each benchmark's 10% budget (out of fold). Random: escalating the same share of tokens at random. The per-route gate uses 18 features; the cascade gate, six confidence features. The last two rows are upper bounds.

Gate	Acc. (%)	Tokens (%)	Random (%)
Code			
Per-route gate	68.0	8.9	67.5
Cascade gate	74.0	10.0	69.0
Perfect draft check	88.3	12.1	–
Oracle router	90.3	8.8	–
Conversations			
Per-route gate	55.0	9.9	53.7
Cascade gate	54.7	9.4	53.0
Perfect draft check	65.7	10.0	–
Oracle router	66.7	3.6	–

compression removes the context cost but leaves the model's own output as the floor.

One caveat qualifies the code numbers. The reference names the right files for 80% of the 201 issues whose gold file was not in its retrieved context at all; the issue text and the model's knowledge of these well-known repositories carry much of the accuracy. Code localisation is therefore a lenient test of memory, and LoCoMo, where answers exist only in the team's conversations, is the strict one.

5.2 The gate: a real signal, a hard budget

The open model's confidence is informative (Table 2). The strongest single feature is whether four resamples repeat the draft's answer; mean token log-probability and the self-check also carry signal, and all are computed from the open model alone, with no closed-model calls. At the 10% budget (Table 3) the cascade gate scores 74.0% on code against 69.0% for random escalation with the same tokens, and keeps 61% of questions on the open model. The per-route gate, despite a higher AUC for its draft score, does worse at the budget (68.0%): it fits 18 features for each of three routes on about 480 training items per fold and decides on the difference of two noisy probabilities. On conversations, where only 38% of drafts are right, a 10% budget already escalates most questions and ranking matters little.

Two upper bounds show why 10% is out of reach for any gate that judges only the draft. A perfect draft check, escalating exactly the wrong drafts, would score 88.3% on code but costs 12.1% of the tokens. The questions a good gate escalates are the expensive ones: the closed model writes 4,543 tokens on questions the open model got wrong against 3,256 on those it got right. Only the oracle passes on code (90.3%

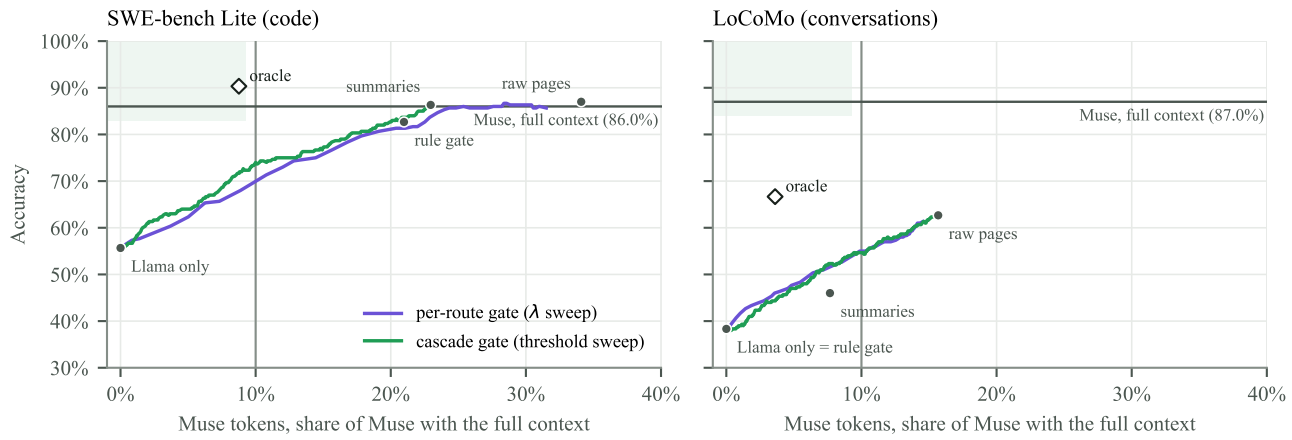


Figure 2: Accuracy against the closed model's tokens. Lines sweep each gate's threshold or λ (out of fold). Dots are fixed routes; the diamond is the oracle router. The shaded box is the pre-registered pass region: within 3 points of the full-context reference at $\leq$10% of its tokens.

Table 4: What 1,000 code questions cost, closed-model tokens and GPU time together, on the same 300 SWE-bench Lite issues; latency is per question. Prices: Muse's list prices; an H100 at $3.95 an hour, charged as if each request had it alone; the memory build included. Raw pages: Muse given the pages behind the retrieved memories, without summaries. Random: as many escalations as the cascade at 10%, mean of 1,000 draws. RouteLLM: an off-the-shelf router [22] deciding from the question alone, before any draft. †Budget chosen after the run.

Option	Acc. (%)	$/1k	$/1k right	Median (s)
Muse, full context	86.0	37.09	43.12	12.2
Muse + raw pages	87.0	18.23	20.96	9.9
Muse + team memory	86.3	16.11	18.66	9.8
Rule gate	82.7	15.25	18.45	8.6
Random escalation	67.5	9.56	14.18	1.1
Cascade, 10%	74.0	11.87	16.04	2.3
Cascade, 20%†	82.7	17.57	21.25	11.0
RouteLLM, 10%	68.3	9.85	14.41	1.3
RouteLLM, 20%†	82.3	14.59	17.72	8.7

at 8.8%), because it also knows when the closed model will fail.

5.3 What a question costs

Token shares leave out two costs: the GPU that runs the open model, and the time a user waits. Table 4 prices each option on the same 300 code questions, from the saved answers; we added this analysis after the run, and it calls no model. Muse is charged its exact tokens at list price. The GPU is charged at Modal's listed H100 rate for each request's open-model time, as if the request had the GPU to itself, which overstates it: the deployment serves up to 32 requests at once. The cascade's confidence calls were not timed and are charged the draft's time again. Every option that reads the memory also pays for building it: 13 minutes of H100 time for these issues, $2.86 per thousand questions, which a team asking more questions of the same code pays once.

Compression does most of the work. Team memory cuts the cost per question by 57% at the same accuracy, and answers slightly sooner. The cascade cuts a further 26% (68% in all), and its median answer arrives in 2.3 s instead of 9.8 s because most questions never leave the GPU; at this budget it scores 74.0% against 86.3%. Per correct answer it costs a little less than memory alone ($16.04 against $18.66 per thousand), and at a 20% budget it costs more than memory alone and is less accurate. Escalating the same number of questions at random is cheaper still but scores 67.5% against the cascade's 74.0%; the cascade pays for that difference with its confidence calls.

The simpler alternatives are close on code. Muse given raw pages, without summaries, costs $18.23 per thousand for 87.0%: once the closed model reasons for thousands of tokens per answer, shortening its prompt further saves little. Our raw-page route retrieves through the memory index; a retriever that indexed raw pages directly would skip the build and cost about as much as team memory, and we did not test one. The rule gate costs about the same as memory alone and is less accurate (82.7% against 86.3%). Two caveats cut the other way. The two runaway answers inflate the reference: without them it costs $29.75 per thousand. And on conversations Muse's own API served 93% of the full-context prompt from its cache; we price those tokens in full, so a provider discount on cached input would make the reference cheaper than the $27.22 per thousand we show (against $7.30 for raw pages and $5.61 for the cascade).

Against an off-the-shelf router. RouteLLM's BERT router [22], trained on Chatbot Arena preferences and used as shipped, decides from the question alone whether the

strong model is needed; we send the questions it picks straight to the closed model with team memory, and set its threshold to the most accurate within each budget on these items, which favours it. Its score barely predicts when the open model's draft is wrong (AUC 0.53 on code, 0.55 on conversations, against the cascade's 0.65 and 0.74). At the 10% budget on code that difference is worth points: the cascade scores 74.0% against the router's 68.3%. The router is cheaper and faster, since it never drafts the questions it escalates. At 20% the two are equally accurate (82.7% and 82.3%), and the router costs less; on conversations at 10% they are level (54.7% and 54.3%). The cascade's advantage is the signal, and it matters most when the budget is tight.

In short, team memory justifies itself on cost alone. The cascade, with an 8B open model, earns its complexity through latency and a lower bill, at some cost in accuracy; it should earn more as the open model answers more questions correctly (Section 6).

5.4 Other open models (added after the run)

To test whether the open model limits the gate, after the run we replaced Llama 3.1 8B with Llama 3.3 70B (FP8, on two H100s) as the model that drafts, and kept everything else fixed: the same 300 code questions, the same team memory (built by the 8B), the closed model's answers from the full run for every escalation, and the same out-of-fold gate training. The 70B's drafts are right on 67.0% of issues against 55.7% (gain +11.3 points, paired bootstrap interval +6.7 to +16.0). The cascade at 10% of the tokens scores 77.7% against 74.0% (gain +3.7, interval −0.7 to +8.0; better in 94% of resamples), still −8.3 points from the reference. At a 20% budget it scores 85.7% (the 8B: 82.7%) at 17.1% of the tokens, keeping 26% of questions on the open model, within 0.3 points of the reference.

A perfect draft check now passes: escalating exactly the 33% of drafts that are wrong would score 90.0% at 9.9% of the tokens. What remains is the gate. Its signal separates the 70B's right and wrong drafts less well than the 8B's (AUC 0.58 against 0.65). Part of this is measurement noise: re-drafting at temperature 0 under batching reproduced the 70B's answer for only 60% of items (the 8B: 89%), so its confidence features describe a slightly different draft from the one scored. The 70B also costs twice the GPU time per request. We report this as a post-hoc experiment, not as meeting the goal.

A simpler gate with a second opinion. Two observations suggest a better judge of the drafts. The self-check alone ranks the 70B's drafts better than the six-signal cascade (AUC 0.69), and so does agreement between the two open models: when the 8B names the same first file as the 70B, the 70B is right far more often. Ranking drafts by the self-check plus one when the 8B agrees, with no fitted weights, gives AUC 0.71 and scores 81.0% at 10% of the tokens (80.3% with the threshold chosen on the other folds), +3.3 points over the cascade (interval −0.3 to +7.0; better in 95% of resamples), and 86.0% at 20%. For the 8B's drafts, with the 70B as the second opinion, the gain is smaller (+1.0, interval −2.7 to +4.7). We designed this rule after looking at these drafts, and it needs both open models running; it marks the direction for the next gate rather than a result. Two further probes did not help. Running the self-check on the exact draft that was scored, rather than a fresh one, left its AUC unchanged (0.69 against 0.69), so the re-drafting noise was not what limited it. A check that shows the model the issue and the team memory's summaries of the file the draft names, and asks whether that file must change, was weaker (AUC 0.60; 0.53 for the 8B), and adding it to the self-check did not raise accuracy at 10% (78.0%). The second opinion is the only signal we found that adds to the self-check. Fitting weights to a single signal does not help here: pooled out-of-fold scores carry each fold's intercept, and folds are whole repositories with different shares of right drafts, so with a signal mostly near zero the fold decides the ranking (`eval/simple_gate.py`).

Muse Glimmer 30B. We then ran Meta's open-weight reasoning model, Muse Glimmer 30B (bf16, one H100), the same way on both benchmarks, at its lowest reasoning strength (Table 5). Two things change for the gate. vLLM returns probabilities for the reasoning as well as the answer, so the gate reads only the answer's tokens, and after reasoning those are nearly certain, so the probability features carry little; the self-check and the resamples reason first. On code, Glimmer's drafts are right on 76.0% of issues, +20.3 points over the 8B (interval +14.7 to +26.0). The cascade at 10% scores 80.3% at 6.8% of the tokens, keeping 77% of questions on the open model (+6.3 over the 8B, interval +1.7 to +11.0), still −5.7 points from the reference; a perfect judge of its drafts would pass (89.0% at 7.5%). On conversations it adds nothing on its own (38.7% against 38.3%; +0.3, interval −3.7 to +4.7), as the retrieval analysis predicts. Reasoning costs time: a draft takes 6.1 s at the median against 1.0 s for the 8B, and on our cost model, which charges each request a whole H100, fast mode costs $21.39 per thousand code questions against $11.87 with the 8B and $16.11 for sending every question to the closed model with memory. Glimmer is the best drafter we measured; at this price it makes fast mode more accurate, not cheaper. Its re-drafts reproduced the scored answer for 81% of code items.

5.5 Sharing across a team

Figure 3 shows the work per engineer as the team grows. With shared memory, pages ingested per engineer fall by 35% at 16 engineers on related tasks, against 13% when each works alone; on random tasks the drop is 7% against 1%. The scoped KV cache behaved as designed on the H100 deployment: with one 6,150-token team memory, a teammate's request reused 6,144 cached prompt tokens and a request from another team reused none. In dollars the ingest saving is small: at the measured build rate, ingest costs $0.0062 per

Table 5: Each open model drafting the same questions, with the same team memory and closed-model answers (all but Llama 3.1 8B added after the run). Alone: the open model with team memory. 10%: the cascade at a 10% budget (its share of Muse's tokens). Perfect: escalating exactly the wrong drafts (share). AUC: the cascade's signal. Fast mode: Muse and GPU dollars per thousand questions, and the median wait. Llama 3.3 70B: FP8 on two H100s, code only. Muse Glimmer 30B: bf16 on one H100, reasoning strength low.

Open model	Alone (%)	10% budget (%)	Perfect check (%)	AUC	Fast mode
Code					
Llama 3.1 8B	55.7	74.0 (10.0)	88.3 (12.1)	0.65	$11.87, 2.3 s
Llama 3.3 70B	67.0	77.7 (9.7)	90.0 (9.9)	0.58	$13.23, 2.6 s
Muse Glimmer 30B	76.0	80.3 (6.8)	89.0 (7.5)	0.64	$21.39, 13.2 s
Conversations					
Llama 3.1 8B	38.3	54.7 (9.4)	65.7 (10.0)	0.74	$5.61, 3.4 s
Muse Glimmer 30B	38.7	57.0 (9.9)	63.3 (9.8)	0.72	$13.81, 10.2 s

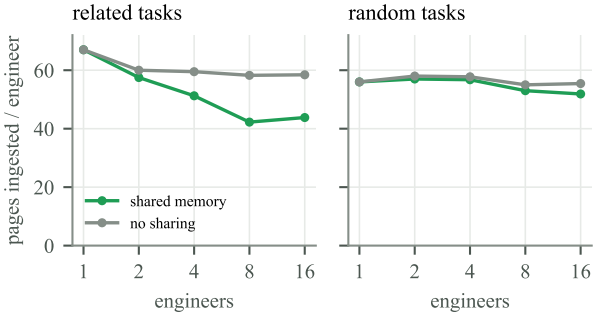


Figure 3: Pages ingested per engineer as the team grows (SWE-bench, one repository, two tasks per engineer).

engineer for two tasks with sharing against $0.0083 alone, against about a cent of closed-model spend per question. Sharing saves work and waiting more than money at this scale. On an H100 the wall-clock saving for 6K tokens is small (0.43 s against 0.59 s, dominated by network round trips); reused tokens are the meaningful measure.

5.6 On conversations, retrieval is the bottleneck

On LoCoMo the best route, raw pages for every question, reaches 62.7% at 15.7% of the tokens, against 87.0%. Splitting items by whether retrieval found a memory from the session that holds the evidence (it did for 63% of questions) locates the loss. When it did, the closed model with raw pages scores 85.3% against the reference's 88.9%; summaries alone score 60.5%, so compression loses detail that raw pages restore. When retrieval missed, every gateway route scores 20.9–23.6% while the reference, which reads everything, scores 83.6%. The oracle router reaches only 66.7%: with this retrieval, no routing policy can pass.

Table 6: The most accurate option within each budget (any fixed route, either gate's sweep). †Budget chosen after seeing the results. ✓: within 3 points of the reference.

Budget	Acc. (gap)	Option	Tokens (%)
Code			
10%	74.0 (−12.0)	cascade gate	10.0
20%†	82.7 (−3.3)	cascade gate	19.9
25%†	86.3 (+0.3) ✓	all to Muse, summaries	23.0
Conversations			
10%	55.0 (−32.0)	per-route gate	9.9
20%†	62.7 (−24.3)	cascade gate	15.6
25%†	62.7 (−24.3)	cascade gate	15.6

5.7 Against the 10% goal

At 10% of the closed model's tokens, the cascade gate reaches 74.0% on code against 86.0% (gap −12.0 points, interval −17.0 to −7.3) and 54.7% on conversations against 87.0%. Neither reaches the pre-registered rule yet. The gate's upper bounds explain the gap on code, and retrieval explains it on conversations; Section 6 sets out the path to the goal. The two subsections that follow report points chosen after the run.

5.8 Larger budgets (chosen after the run)

A team may prefer accuracy over the deepest saving. Table 6 reports 20% and 25% budgets, which we chose only after seeing the results and present as points on the curve, not as a pass. On code, sending every question with summaries matches the reference at 23.0% of its tokens. Two of the reference's answers reasoned to the closed model's 262,144-token output limit and returned nothing; they are 11% of its tokens, so we describe this point as about a quarter of the tokens. At that budget most of the saving comes from compact memory, not from answering locally. Conversations stop at 62.7% whatever the budget.

5.9 Lower reasoning effort (chosen after the run)

Because the closed model's output dominates escalation cost, we reran every closed-model condition at its minimal reasoning effort on the same memory and drafts, together with a full-context reference at the same effort (Table 7). Output per escalation falls from 2,822 to 336 tokens. On code, the per-route gate then scores 80.7% against 83.0% (gap −2.3, interval −5.3 to +0.7) at 9.8% of the tokens, keeping 13% of questions on the open model. Choosing λ on other folds gives a similar result, so in-sample selection does not explain it. The point estimate lies inside the pass rule, but the interval admits misses of over 5 points (the gap is at least −3 in 67% of resamples), the setting was chosen after the run, and the reference itself is weaker than at default effort; against the default-effort reference the same gate misses by 5 points. Notably, sending every question with summaries at minimal effort scores 83.3%, a gap of −2.7 points to the *default-effort* reference, at 8.0% of its tokens. Conversations do not move (57.3%). We report this as a post-hoc finding and do not count it as meeting the goal.

6 Discussion

Where the cost goes. The study began from the premise that context dominates the bill. For a reasoning model that premise holds only until the context is compressed: after that, the closed model's own reasoning is most of each escalation. A 10% total-token budget then implies escalating only a few questions, and the gate has to be both accurate and selective. Two levers follow: send fewer questions (a better gate), or make each escalation cheaper (lower reasoning effort, which in our post-hoc run moved code close to the bar).

What a gate can and cannot know. The open model's confidence identifies its own wrong answers reasonably well, but the oracle's advantage comes from knowing when the closed model will also fail, which no draft-only signal provides. Routing work that reports only the cheap model's accuracy, or counts calls rather than tokens, may understate this limit.

The path to the goal. The goal remains Itara's target, and the measurements suggest it is within reach with more resources than a nine-day study on one GPU. Three observations support this on code. First, the pass region is not empty: the oracle router reaches 90.3% at 8.8% of the tokens. Second, the limiting factor is the open model. The 8B model's draft is right on only 56% of issues, so even a perfect draft check, escalating exactly the wrong drafts, costs 12.1% of the tokens, just over the budget. We tested this after the run (Section 5.4): with Llama 3.3 70B the drafts are right on 67.0% of issues, and a perfect draft check passes the goal (90.0% at 9.9% of the tokens). Larger models in the same family score substantially higher [18] and are often better calibrated about their own answers [11]; in our run, though, the gate's signal was weaker for the 70B, so a better judge of the drafts is now the main lever. Third, the gate was trained on about 480 items per fold; more questions, which a deployed gateway collects as a matter of course, should help the per-route gate most, as it overfits at this size. Lower reasoning effort already brought code within about 2 points at 10% (Table 7). The 70B moved the cascade from 74.0% to 77.7% at 10% of the tokens, and Muse Glimmer to 80.3% (at 6.8%); closing the rest needs a better draft judge more than a bigger model. On conversations a larger model is not enough: retrieval caps every route, so better retrieval comes first. We present these as expectations to be tested, not results.

Why open weights. The shared cache requires control of the model's attention state, which closed APIs do not expose, and a cache is only reusable by the model that produced it. The layer that processes a team's repeated reading is therefore an open-weight model by necessity; Llama is what we built and measured on. Other open families would serve as well technically.

7 Limitations and threats to validity

- **Simulated teams.** Teams are repositories and conversations, and the replay simulates engineers; we did not observe real teams.
- **One closed model and two open-model families** (Llama 3.1 8B; after the run, Llama 3.3 70B on code and Muse Glimmer 30B); results may differ for others.
- **An LLM judge** scores LoCoMo. The hand check agreed on 48 of 50 verdicts, both disagreements being lenient judge calls.
- **Benchmark leniency.** Code localisation on well-known repositories rewards prior knowledge (the reference is often right without seeing the file).
- **Pooled out-of-fold scores.** The gates are scored by pooling five folds' predictions under one threshold. Folds are whole repositories, and their intercepts differ, which can blur the ranking (Section 5.4); a deployed gate is one model and does not have this problem.
- **Post-hoc choices** (budgets, reasoning effort) are reported separately and with intervals, but they were made after seeing results. Choosing the operating point from a sweep on the same items is a mild selection; our nested check suggests it is small here.
- **Sample size.** With 300 items per benchmark, intervals on gaps are about ±3 points.
- **The cost model** is ours: list prices, an H100 charged per request as if unshared, the confidence calls' time estimated rather than measured, and no discount for the closed model's own cached input. We compare against simpler compression and routing (raw pages, a rule gate, random escalation, an off-the-shelf router) but not against an off-the-shelf prompt compressor [9] or plain retrieval over raw pages.

Table 7: Code, with the closed model at minimal reasoning effort (a setting chosen after the results; same memory, drafts and probes). Gaps against the full-context reference at the same effort (83.0%) and at default effort (86.0%), with paired bootstrap 95% intervals. "Other folds": each fold's λ is chosen on the other folds, removing in-sample selection.

		vs. reference, same effort		vs. reference, default effort	
Option	Acc. (%)	Gap [95% CI]	Tokens (%)	Gap [95% CI]	Tokens (%)
Per-route gate, λ chosen in-sample	80.7	−2.3 [−5.3, +0.7]	9.8	−5.3 [−8.7, −2.3]	6.8
Per-route gate, λ chosen on other folds	81.0	−2.0 [−5.0, +0.7]	9.9	−5.0 [−8.3, −2.0]	6.8
Cascade gate	79.7	−3.3 [−6.7, −0.3]	10.0	−6.3 [−9.7, −3.0]	6.9
Always escalate, summaries	83.3	+0.3 [−2.0, +2.7]	11.6	−2.7 [−5.3, +0.0]	8.0

- **Cache scope.** We tested scope-keyed prefix caching on one server; cross-server sharing (LMCache) was not evaluated.

8 Conclusion

Itara shares one open-weight model, one compressed memory and one permission-scoped KV cache across a team, and escalates to a closed model only when the open model is unsure. On code, compact team memory lets the closed model match its full-context accuracy with under a tenth of the context and about a quarter of the tokens; the open model's confidence is a usable escalation signal; and sharing memory makes each extra engineer cheaper while the scoped cache keeps teams apart. On conversations, retrieval rather than routing limits accuracy. The ambitious goal we set in advance, matching the closed model at 10% of its tokens, is not reached yet, but the measurements point to it: the oracle passes on code, the open model's accuracy is the main limit, and at minimal reasoning effort the gate comes within about 2 points. Stronger open models, tested after the run, lift the cascade at 10% from 74.0% to 77.7% (Llama 3.3 70B) and 80.3% (Muse Glimmer 30B, with slower, reasoning drafts) and make a perfect draft check pass. Future versions of Itara aim at the goal directly, with a larger open model on more GPUs, gates trained on more questions, better retrieval, cheaper escalations and gates that also anticipate the closed model's failures. Every escalation pairs the open model's draft with the closed model's answer on the team's own questions, which also makes the gateway a natural source of data for improving the team's open model.

Reproducibility

Code, per-item answers for every real-model run, gate replays and analysis notebooks are in the repository (`results/`). All tables and figures in this paper are generated from those files by `paper/generate.py`, which calls no model. Simulated runs used to check the pipeline are excluded from all reported numbers.

Disclosure and use of AI assistance

This work was carried out during the Meta Global AI Developer Hackathon (26 October–4 November 2026). The only earlier code in use, about 20 lines of a compression prompt from the author's earlier project, was imported in one marked commit; everything else was written during the event (see `DISCLOSURE.md`). AI coding assistants were used for research, planning, code and writing; all design decisions, experiments and results were reviewed by the author. Benchmark data are public (SWE-bench Lite, LoCoMo) and no private or company data was used.

References

[1] Lingjiao Chen, Matei Zaharia, and James Zou. FrugalGPT: How to use large language models while reducing cost and improving performance. *arXiv preprint arXiv:2305.05176*, 2023.

[2] Gordon V. Cormack, Charles L. A. Clarke, and Stefan Büttcher. Reciprocal rank fusion outperforms Condorcet and individual rank learning methods. In *Proceedings of the 32nd International ACM SIGIR Conference on Research and Development in Information Retrieval*, 2009.

[3] Dujian Ding, Ankur Mallick, Chi Wang, Robert Sim, Subhabrata Mukherjee, Victor Rühle, Laks V. S. Lakshmanan, and Ahmed Hassan Awadallah. Hybrid LLM: Cost-efficient and quality-aware query routing. In *International Conference on Learning Representations (ICLR)*, 2024.

[4] Bradley Efron and Robert J. Tibshirani. *An Introduction to the Bootstrap*. Chapman & Hall, 1993.

[5] In Gim, Guojun Chen, Seung-seob Lee, Nikhil Sarda, Anurag Khandelwal, and Lin Zhong. Prompt cache: Modular attention reuse for low-latency inference. In *Proceedings of Machine Learning and Systems (MLSys)*, 2024.

[6] Chenchen Gu, Xiang Lisa Li, Rohith Kuditipudi, Percy Liang, and Tatsunori Hashimoto. Auditing prompt caching in language model APIs. In *Proceedings of the*

42nd International Conference on Machine Learning (ICML), volume 267 of *PMLR*, pages 20477–20496, 2025.

[7] Neha Gupta, Harikrishna Narasimhan, Wittawat Jitkrittum, Ankit Singh Rawat, Aditya Krishna Menon, and Sanjiv Kumar. Language model cascades: Token-level uncertainty and beyond. In *International Conference on Learning Representations (ICLR)*, 2024.

[8] Kelly Hong, Anton Troynikov, and Jeff Huber. Context rot: How increasing input tokens impacts LLM performance. Technical report, Chroma, 2025. URL `https://research.trychroma.com/context-rot`.

[9] Huiqiang Jiang, Qianhui Wu, Chin-Yew Lin, Yuqing Yang, and Lili Qiu. LLMLingua: Compressing prompts for accelerated inference of large language models. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2023.

[10] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *International Conference on Learning Representations (ICLR)*, 2024.

[11] Saurav Kadavath, Tom Conerly, Amanda Askell, et al. Language models (mostly) know what they know. *arXiv preprint arXiv:2207.05221*, 2022.

[12] Andrew Kane. pgvector: Open-source vector similarity search for Postgres. `https://github.com/pgvector/pgvector`, 2021.

[13] Lorenz Kuhn, Yarin Gal, and Sebastian Farquhar. Semantic uncertainty: Linguistic invariances for uncertainty estimation in natural language generation. In *International Conference on Learning Representations (ICLR)*, 2023.

[14] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP)*, 2023.

[15] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

[16] Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics*, 12, 2024.

[17] Yuhan Liu, Yihua Cheng, Jiayi Yao, Yuwei An, Xiaokun Chen, Shaoting Feng, Yuyang Huang, Samuel Shen, Rui Zhang, Kuntai Du, and Junchen Jiang. LMCache: An efficient KV cache layer for enterprise-scale LLM inference. *arXiv preprint arXiv:2510.09665*, 2025. MLSys 2026.

[18] Llama Team, AI @ Meta. The Llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024.

[19] Adyasha Maharana, Dong-Ho Lee, Sergey Tulyakov, Mohit Bansal, Francesco Barbieri, and Yuwei Fang. Evaluating very long-term conversational memory of LLM agents. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2024.

[20] Potsawee Manakul, Adian Liusie, and Mark J. F. Gales. SelfCheckGPT: Zero-resource black-box hallucination detection for generative large language models. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2023.

[21] Meta. Meta Model API (Muse Spark). Accessed through the Meta Global AI Developer Hackathon, 2026.

[22] Isaac Ong, Amjad Almahairi, Vincent Wu, Wei-Lin Chiang, Tianhao Wu, Joseph E. Gonzalez, M. Waleed Kadous, and Ion Stoica. RouteLLM: Learning to route LLMs from preference data. In *International Conference on Learning Representations (ICLR)*, 2025.

[23] Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. MemGPT: Towards LLMs as operating systems. *arXiv preprint arXiv:2310.08560*, 2023.

[24] Linke Song, Zixuan Pang, Wenhao Wang, Zihao Wang, XiaoFeng Wang, Hongbo Chen, Wei Song, Yier Jin, Dan Meng, and Rui Hou. The early bird catches the leak: Unveiling timing side channels in LLM serving systems. *arXiv preprint arXiv:2409.20002*, 2024.

[25] The PostgreSQL Global Development Group. PostgreSQL documentation: Row security policies. `https://www.postgresql.org/docs/current/ddl-rowsecurity.html`, 2024.

[26] Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc Le, Ed Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. Self-consistency improves chain of thought reasoning in language models. In *International Conference on Learning Representations (ICLR)*, 2023.

[27] Chunqiu Steven Xia, Yinlin Deng, Soren Dunn, and Lingming Zhang. Agentless: Demystifying LLM-based software engineering agents. *arXiv preprint arXiv:2407.01489*, 2024.

[28] Shitao Xiao, Zheng Liu, Peitian Zhang, and Niklas Muennighoff. C-Pack: Packaged resources to advance general Chinese embedding. *arXiv preprint arXiv:2309.07597*, 2023.

[29] Jiayi Yao, Hanchen Li, Yuhan Liu, Siddhant Ray, Yihua Cheng, Qizheng Zhang, Kuntai Du, Shan Lu, and Junchen Jiang. CacheBlend: Fast large language model serving for RAG with cached knowledge fusion. In *Proceedings of the Twentieth European Conference on Computer Systems (EuroSys)*, pages 94–109, 2025.

[30] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. Judging LLM-as-a-judge with MT-Bench and Chatbot Arena. In *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, 2023.

[31] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E. Gonzalez, Clark Barrett, and Ying Sheng. SGLang: Efficient execution of structured language model programs. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.